FACULTAD DE INGENIERÍA – FAIN

Departamento de Tecnología Informática – DEINI

Fundamentos de Informática – Comisión Viernes Noche · 2do Cuatrimestre 2026

TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Inventario

Fechas clave

Instancia	Clase	Fecha	Detalle
Entrega de la consigna y armado de grupos	Clase 9	02/10	Grupos de 2 a 3 integrantes
Avance 1: matriz, menú, alta y carga desde CSV	Clase 10	09/10	Revisión del avance en clase
Avance 2: búsqueda secuencial y binaria	Clase 11	16/10	Revisión del avance en clase
Avance 3: métodos de ordenamiento	Clase 12	23/10	Revisión del avance en clase
Consultas sobre el TP	Clase 13	26/10	Clase virtual sincrónica
Avance 4: baja, modificación, reportes y guardado	Clase 14	30/10	Pre-entrega: el sistema completo funcionando
Entrega final del TP	—	**05/11**	Por el campus virtual, hasta las 23:59
Chequeo del TP con el docente	Clase 15	06/11	Resultado: Aprobado / No aprobado
Segundo Parcial (individual, en computadora)	Clase 16	**13/11**	Basado en el TP
Reentrega (solo TP no aprobados)	Clase 17	20/11	Con las correcciones indicadas
Coloquio final: defensa individual del TP	Clase 18 / 20	**27/11 o 11/12**	Fecha asignada por sorteo

Modalidad

- El TP se desarrolla en **grupos de 2 a 3 integrantes**. Excepcionalmente individual, con autorización previa del docente.
- La calificación del TP es **Aprobado / No aprobado**. Tener el TP aprobado (chequeado con el docente) es **condición obligatoria para rendir el final**.
- El TP es grupal, pero **todas las evaluaciones que se basan en él son individuales**: el Segundo Parcial y el coloquio final.
- Cada integrante debe conocer y poder explicar y modificar **todo** el código del grupo, no solo la parte que programó. En el coloquio final cada alumno defiende el TP **solo**, y los integrantes de un mismo grupo pueden quedar asignados a fechas distintas.

Objetivo general

Aplicar de manera integrada los contenidos de la materia — estructuras de control, funciones, listas y matrices, y los algoritmos de búsqueda y ordenamiento trabajados en las Clases 11 y 12 — en el desarrollo de un programa completo en Python que resuelva un problema real de gestión de datos.

Consigna

Desarrollar en Python un **Sistema de Gestión de Inventario** para un pequeño comercio, que permita cargar, consultar, modificar y analizar el stock de productos desde la consola.

Estructura de datos (obligatoria)

El inventario se guarda en una **matriz (lista de listas)**, donde cada fila es un producto. Las columnas van **siempre en este orden**:

Columna	Campo	Tipo	Ejemplo
0	Código (único)	entero	`101`
1	Nombre	texto	`"Yerba 1kg"`
2	Categoría	texto	`"Almacén"`
3	Precio unitario	número	`3500.0`
4	Stock actual	entero	`12`

El archivo `inventario.csv` tiene una primera línea de encabezado y un producto por línea, con los campos separados por coma y en el mismo orden:

```
codigo,nombre,categoria,precio,stock
101,Yerba 1kg,Almacén,3500.0,12
102,Detergente 750ml,Limpieza,1800.0,4
```

> **¿Por qué es obligatoria?** El Segundo Parcial se basa en el TP: los ejercicios usan esta misma matriz y estas mismas funciones. Respetar la estructura y los nombres de las funciones es lo que permite que el parcial sea igual para todos.

Funciones obligatorias

Cada función debe tener **exactamente** este nombre y estos parámetros:

Función	Qué hace / qué devuelve
`cargar_csv(nombre_archivo)`	Lee el archivo y devuelve la matriz de productos.
`guardar_csv(matriz, nombre_archivo)`	Escribe la matriz en el archivo, con encabezado.
`alta_producto(matriz, codigo, nombre, categoria, precio, stock)`	Agrega el producto. Devuelve `True` si lo agregó, `False` si el código ya existe o los datos no son válidos.
`baja_producto(matriz, codigo)`	Elimina el producto. Devuelve `True` si lo eliminó, `False` si no existe.
`modificar_stock(matriz, codigo, cantidad)`	Suma la cantidad (positiva o negativa) al stock. Devuelve `False` si el producto no existe o si el stock quedaría negativo.
`busqueda_secuencial(matriz, codigo)`	Devuelve dos valores: el índice de la fila (o `-1` si no existe) y la cantidad de comparaciones realizadas.
`busqueda_binaria(matriz, codigo)`	Igual que la anterior, sobre la matriz ordenada por código.

`ordenar_seleccion(matriz, columna)`
`ordenar_burbuja(matriz, columna)`
`ordenar_insercion(matriz, columna)`
	Implementar **al menos dos** de las tres. Ordenan la matriz de menor a mayor según la columna indicada y la devuelven.

`stock_bajo(matriz, umbral)`	Devuelve una lista con los productos cuyo stock es menor al umbral.
`valor_total(matriz)`	Devuelve la suma de precio × stock de todos los productos.

`producto_mas_caro(matriz)` `producto_mas_barato(matriz)`	Devuelven la fila del producto correspondiente.

Además de estas, el grupo puede crear todas las funciones auxiliares que necesite (por ejemplo, para mostrar la matriz o pedir datos al usuario).

Funcionalidades del programa

1. **Menú principal:** un menú de texto con las opciones del sistema, que se repita hasta que el usuario decida salir (estructura iterativa + condicional).
2. **Carga inicial:** al iniciar, leer `inventario.csv` y cargar los productos en la matriz.
3. **Alta, baja y modificación de stock** de productos, validando los datos ingresados.
4. **Búsqueda por código** con los dos métodos, informando cuántas comparaciones hizo cada uno para poder compararlos (Clase 11).
5. **Ordenamiento** por Nombre, Precio o Stock, a elección del usuario (Clase 12).
6. **Reportes:** listado completo ordenado según el criterio elegido; productos con stock bajo (umbral ingresado por el usuario); valor total del inventario; producto más caro y más barato.
7. **Guardar cambios:** exportar el estado final del inventario a `inventario.csv` antes de salir.

Desafíos opcionales

No cambian la condición de Aprobado, pero son buen material para lucirse en el coloquio:

- Interfaz gráfica simple con tkinter.
- Exportación adicional en formato JSON.
- Manejo de excepciones (try/except) ante datos inválidos o archivos inexistentes.

Requisitos técnicos

- Cada operación se implementa como una **función con parámetros y valor de retorno**. No se acepta lógica repetida "suelta" en el programa principal.
- Las búsquedas, los ordenamientos y el cálculo de máximo y mínimo se programan **a mano**, con los algoritmos vistos en clase. No se permite usar `sort()`, `sorted()`, `min()`, `max()`, `index()` ni el operador `in` para resolverlos.
- Validar con estructuras condicionales los datos que ingresa el usuario (por ejemplo, evitar precios o stock negativos).
- Nombres de variables y funciones claros y consistentes, y código comentado explicando qué hace cada función.
- Se recomienda usar solo las herramientas del lenguaje vistas en la materia. Cualquier otra construcción que aparezca en el código puede ser preguntada en el parcial y en el coloquio.

Uso de inteligencia artificial

Durante el desarrollo del TP se puede usar IA como herramienta de apoyo (para consultar dudas, entender errores o revisar el código), con estas condiciones:

- Se debe **declarar en el informe** qué herramienta se usó y para qué.
- Todo el código entregado debe poder ser explicado y modificado por **cada** integrante, sin ayuda. Ese es justamente el contenido del Segundo Parcial y del coloquio final.
- Entregar código que el alumno no puede explicar se considera falta de autoría.

Entregables

Un integrante por grupo sube al campus virtual, hasta el **05/11 a las 23:59**, un archivo `TP_GrupoN.zip` con:

1. El código fuente (.py), comentado y funcionando.
2. El archivo `inventario.csv` con al menos 15 productos de prueba.

3. Un informe breve (máximo 3 páginas) con: integrantes; estructura de datos; descripción de las funciones principales; análisis de qué método de búsqueda y de ordenamiento conviene usar en este caso y por qué; cómo se repartieron el trabajo; y declaración del uso de IA.

Condiciones de aprobación del TP

El TP se chequea con el docente en la Clase 15 (06/11). Para estar **Aprobado** debe cumplir **todos** estos puntos:

Criterio	Cumple
El programa se ejecuta sin errores de principio a fin.	Sí / No
Respeta la estructura de la matriz y los nombres y parámetros de las funciones obligatorias.	Sí / No
Carga y guarda correctamente el archivo CSV.	Sí / No
Alta, baja y modificación de stock funcionan y validan los datos.	Sí / No
Búsqueda secuencial y binaria implementadas a mano, informando las comparaciones.	Sí / No
Al menos dos métodos de ordenamiento implementados a mano.	Sí / No
Los cuatro reportes funcionan correctamente.	Sí / No
Uso correcto de funciones (parámetros y retorno), sin lógica repetida.	Sí / No
Código comentado e informe completo.	Sí / No

Si el TP resulta **No aprobado**, el grupo recibe las observaciones y tiene **una reentrega hasta el 20/11** (Clase 17). Sin el TP aprobado no se puede rendir el final.

Cómo se evalúa a partir del TP

Segundo Parcial — 13/11 (individual, en computadora)

Se rinde en computadora y trabaja sobre la estructura y las funciones del TP. Incluye:

- Un par de **preguntas teóricas** sobre los conceptos del TP.
- **Tres ejercicios sin IA**: escribir o modificar funciones sobre la matriz del inventario. Solo en blog de notas o IDLE
- **Un ejercicio con IA**: se resuelve con ayuda de una herramienta de IA

Se **graba la pantalla** durante su resolución. La grabación solo se revisa si se detecta una posible deshonestidad académica. Conviene probar el programa de grabación antes del parcial.

Se aprueba con 4 (cuatro) o más. El recuperatorio es el 27/11.

Coloquio final — 27/11 o 11/12 (individual)

- Cada alumno defiende el TP de su grupo **de forma individual**. La fecha de cada alumno se asigna por **sorteo**: los integrantes de un mismo grupo pueden rendir en fechas distintas.
- El docente pide explicar partes del código y hacer **modificaciones en vivo, sin IA**, sobre las funciones del TP.
- Para rendir hay que tener aprobados el TP, el Primer Parcial y el Segundo Parcial (o sus recuperatorios).

Honestidad académica

El trabajo debe ser de autoría del grupo, y cada integrante es responsable de entender todo lo que se entrega. Los actos de deshonestidad académica o cualquier situación de indisciplina serán sancionados según el régimen disciplinario correspondiente.